BIG DATA TECHNOLOGIES

Comprehensive Cheatsheet for CCEE Exam
Sessions 12–19

Study Guide

January 2026

Contents

1 SESSION 12: HADOOP ETL DEVELOPMENT **3**
- 1.1 ETL Process Overview . . . 3
- 1.2 ELT (Extract–Load–Transform) Pattern . . . 3
- 1.3 Typical ETL Functions . . . 3
- 1.4 Data Extraction in Hadoop . . . 3
- 1.5 Why Use ETL Tools (vs hand-coded MapReduce) . . . 3
- 1.6 Advantages of ETL Tools on Hadoop/Cloud . . . 3

2 SESSION 13: HBASE INTRODUCTION **4**
- 2.1 HBase Overview . . . 4
- 2.2 Data Model (Key Terms for MCQs) . . . 4
- 2.3 HBase Architecture . . . 4
- 2.4 Installation Conceptual Steps . . . 4
- 2.5 Installation Modes . . . 4

3 SESSION 14: HBASE ADMIN, OPERATIONS & SECURITY **5**
- 3.1 HBase Shell: Administrative (DDL) Commands . . . 5
- 3.2 HBase Shell: Data (CRUD/DML) Commands . . . 5
- 3.3 Java Client API (org.apache.hadoop.hbase.client) – DML/CRUD . . . 5
- 3.4 HBase Admin API (HBaseAdmin / Admin) – DDL . . . 5
- 3.5 HBase Security . . . 5

4 SESSION 16: HIVE DATA WAREHOUSE – INTRODUCTION **6**
- 4.1 Hive Overview . . . 6
- 4.2 Hive Architecture Components . . . 6
- 4.3 Hive Installation (Conceptual) . . . 6
- 4.4 Hive vs Traditional RDBMS . . . 6

5 SESSION 17: WORKING WITH HIVE QL **7**
- 5.1 Datatypes . . . 7
- 5.2 Operators & Built-in Functions . . . 7
- 5.3 Managed (Internal) Tables . . . 7
- 5.4 External Tables . . . 7
- 5.5 Partitions . . . 7
- 5.6 Buckets . . . 8
- 5.7 Storage Formats . . . 8
- 5.8 Importing Data (DML Operations) . . . 8
- 5.9 Altering Tables (DDL) . . . 8
- 5.10 Dropping Tables . . . 8

6 SESSION 18: QUERYING WITH HIVE QL **9**
6.1 Sorting Clauses . . . 9
6.2 Aggregation with GROUP BY . . . 9
6.3 Joins in HiveQL . . . 9
6.4 Subqueries . . . 9
6.5 Views . . . 9
6.6 MapReduce Scripts (TRANSFORM Clause) . . . 10

7 SESSION 19: MORE ON HIVE QL (DATA MANIPULATION, UDFS, SCRIPTS) **11**
7.1 Data Manipulation (DML) . . . 11
7.2 User-Defined Functions (UDFs) . . . 11
7.3 Custom MapReduce in Hive . . . 11
7.4 Appending Data into Existing Tables . . . 11
7.5 Writing HQL Scripts (.hql Files) . . . 11
7.6 Common HiveQL SELECT Clauses (Summary) . . . 12

8 KEY MCQ TRAPS & MNEMONICS **13**
8.1 ETL vs ELT . . . 13
8.2 Hive vs RDBMS in One Sentence . . . 13
8.3 Managed vs External Table on DROP . . . 13
8.4 Partitions vs Buckets . . . 13
8.5 Reduce-side vs Map-side Join . . . 13
8.6 Sorting Clauses Ranking (Performance Impact) . . . 13
8.7 Hive Operations Cheat . . . 13

9 QUICK REFERENCE: ESSENTIAL COMMANDS **14**
9.1 HBase Shell (Most Common) . . . 14
9.2 Hive QL (Most Common) . . . 14
9.3 Hive MapReduce Script Integration . . . 14
9.4 Execute HQL Script . . . 14

10 EXAM STRATEGY REMINDERS **15**

11 FINAL SUMMARY TABLE **16**

1 SESSION 12: HADOOP ETL DEVELOPMENT

1.1 ETL Process Overview

ETL = Extract → Transform → Load

- **Extract:** Pull data from OLTP DBs, logs, CSV/JSON files, APIs, message queues → HDFS landing zone
- **Transform:** Clean (remove nulls, fix formats), enrich (lookups, joins), aggregate, filter, deduplicate, validate using MapReduce/Hive/Pig/Spark
- **Load:** Store final data into Hive tables, HBase, NoSQL, or downstream RDBMS via connectors (Sqoop)

1.2 ELT (Extract–Load–Transform) Pattern

Load all raw data into Hadoop data lake first, then build multiple views/data marts later. Common in big data: flexibility to transform data on-demand for different use cases.

1.3 Typical ETL Functions

- Data cleaning: handle missing values, invalid codes, type conversions
- Data integration: joins across sources, key mapping
- Data transformation: derived fields (qty × price), aggregations (SUM, COUNT, AVG, MIN, MAX), sorting, filtering, normalization
- Data quality: deduplication, constraint checks, validation rules

1.4 Data Extraction in Hadoop

- From relational DBs: **Sqoop** or custom connectors/export jobs
- From files/logs: **Flume, Kafka, custom ingestion** to HDFS
- From streams: Kafka + Spark/Flink for near real-time ETL

1.5 Why Use ETL Tools (vs hand-coded MapReduce)

- **Productivity**: Drag–drop or SQL-based design instead of low-level code
- **Reusability & maintenance**: Centralized job definitions, versioning, documentation
- **Orchestration**: Workflows with dependencies, retries, scheduling, logging, alerts
- **Data quality**: Built-in profiling, validation, error-handling
- **Connectivity**: Ready-made connectors for DBs, SaaS, files, streams, cloud storage

1.6 Advantages of ETL Tools on Hadoop/Cloud

- **Automation**: Scheduling, monitoring, automatic reruns reduce errors
- **Scalability**: Leverage Hadoop/Spark, scale with cluster size
- **Performance**: Parallel data flows, pushdown optimization, cluster-aware execution
- **Cost-effectiveness**: Commodity hardware + open-source stack; pay-as-you-go cloud models
- **Flexibility**: Support structured, semi-structured, unstructured data (CSV, JSON, logs, images, etc.)

2 SESSION 13: HBASE INTRODUCTION

2.1 HBase Overview

Distributed, column-oriented NoSQL database on HDFS. For fast random reads/writes (online access), not batch/stream like HDFS. Ideal for time-series data, clickstreams, IoT metrics, user profiles, random read/write with high throughput.

2.2 Data Model (Key Terms for MCQs)

- **Table** → rows
- **Row key** → sorted lexicographically; unique identifier for each row
- **Column family** → logical group of columns; defined at schema level; stored together on disk
- **Column qualifier** → specific column inside a family (dynamic, can vary per row)
- **Cell** = (row key, column family, column qualifier, timestamp) → value
- **Versions** → identified by timestamp; each cell can have multiple timestamped versions

2.3 HBase Architecture

- **HMaster**: Lightweight master managing cluster, assigning regions to RegionServers, DDL (create/alter/delete), load balancing
- **RegionServer (HRegionServer)**: Worker serving read/write requests for assigned regions
  - **MemStore**: Write cache in RAM
  - **HFile**: On-disk store in HDFS
  - **BlockCache**: Read cache
  - **WAL (Write-Ahead Log)**: Durability log
- **ZooKeeper**: Coordination service for master election, server registration, metadata discovery by clients
- **Regions**: Horizontal partitions (row-key ranges) of a table
  - Each region served by one RegionServer at a time
  - Automatically split when too large for load distribution

2.4 Installation Conceptual Steps

1. **Prerequisites**: Working Hadoop cluster (HDFS), ZooKeeper ensemble
2. **Configuration**: Download HBase, configure `hbase-site.xml` pointing to HDFS and ZooKeeper
3. **Start services**: Start ZooKeeper, HMaster, RegionServers
4. **Verify**: Use HBase shell (`create`, `list`, `put`, `get`, `scan`) to verify

2.5 Installation Modes

- **Standalone**: Single JVM, local file system (dev only)
- **Pseudo-distributed**: All daemons on one machine with HDFS (testing)
- **Fully distributed**: Multi-node cluster with HDFS and ZooKeeper (production)

3 SESSION 14: HBASE ADMIN, OPERATIONS & SECURITY

3.1 HBase Shell: Administrative (DDL) Commands

- `create 't1', 'cf1','cf2'` – Create table with column families
- `disable 't1'`, `enable 't1'` – Change table state
- `alter 't1', {NAME => 'cf1', VERSIONS => 3}` – Modify schema
- `describe 't1'` – Show table schema
- `drop 't1'` – Delete table (removes metadata and data)

3.2 HBase Shell: Data (CRUD/DML) Commands

- `put 't1','row1','cf1:col1','value1'` – Create/Update
- `get 't1','row1'` – Read one row
- `scan 't1'` – Read many rows (can specify filters, columns, LIMIT)
- `delete 't1','row1','cf1:col1'` or `deleteall 't1','row1'` – Delete
- `count 't1'` – Count rows
- `truncate 't1'` – Disable, drop, recreate table (clears data, keeps structure)

3.3 Java Client API (org.apache.hadoop.hbase.client) – DML/CRUD

- `Connection` / `ConnectionFactory` – Create cluster connection
- `Table` – Logical handle to HBase table
- `Put` – Create/update row
- `Get` – Retrieve row or specific columns
- `Delete` – Delete columns/rows
- `Scan` – Iterate over ranges of rows
- `Result` – Represents data returned from Get/Scan

3.4 HBase Admin API (HBaseAdmin / Admin) – DDL

- `admin.createTable(...)` – Create table
- Disable/enable table
- Add/modify/delete column families
- Get table descriptors, list tables

3.5 HBase Security

- **Authentication**: **Kerberos-based** with SASL at RPC level; only authenticated users/clients can communicate with HBase
- **Authorization**: Implemented via **Access Controller / ACL coprocessor**
  - Grants permissions: R (Read), W (Write), X (Execute), C (Create), A (Admin) at table/-family/qualifier level
  - `grant 'user','RWXCA','table','cf','col'` – Grant permissions
  - `revoke 'user','table'` – Remove permissions

4 SESSION 16: HIVE DATA WAREHOUSE – INTRODUCTION

4.1 Hive Overview

Data warehouse system on Hadoop using SQL-like queries (HiveQL) over HDFS. For **batch analytics** (OLAP), not OLTP. Created at Facebook to let analysts write SQL over massive log data.

4.2 Hive Architecture Components

- **HiveQL**: SQL-like query language
- **Driver**: Receives query, manages session, coordinates execution (parser + planner + optimizer)
- **Compiler**: Parses query, performs semantic analysis, generates execution plan
- **Metastore**: RDBMS-backed metadata store (table schemas, partitions, locations, SerDe info)
- **Execution engine**: Converts plan to jobs on Hadoop: MapReduce / Tez / Spark
- **HiveServer2 / Clients**: JDBC/ODBC/CLI interfaces for apps and users

4.3 Hive Installation (Conceptual)

1. Install Hadoop (HDFS/YARN)
2. Install Hive binaries
3. Configure metastore DB (MySQL/PostgreSQL/Derby) and `hive-site.xml`
4. Start metastore service and HiveServer2
5. Connect via CLI or Beeline; run HiveQL queries

4.4 Hive vs Traditional RDBMS

Aspect	**HIVE**	**RDBMS**
Workload	OLAP/batch; scans huge datasets	OLTP; many small reads/writes
Latency	High (batch); startup cost	Low; interactive
Storage	HDFS; horizontal scale to TB–PB	Local/RAID; vertical scale; indexes
Schema	Schema-on-read; data dumped first	Schema-on-write; schema enforced at insert
Execution	Distributed jobs (MR/Tez/Spark)	Single server/-cluster, low-latency engine
Transactions	No strict row-level ACID	Strict ACID by default

5 SESSION 17: WORKING WITH HIVE QL

5.1 Datatypes

Primitive Types:

- Numeric: TINYINT, SMALLINT, INT, BIGINT, FLOAT, DOUBLE, DECIMAL
- String: STRING, VARCHAR, CHAR
- Date/time: DATE, TIMESTAMP
- Boolean: BOOLEAN

Complex Types:

- ARRAY<type>
- MAP<key_type, value_type>
- STRUCT<field1:type1, field2:type2, ...>
- UNIONTYPE

5.2 Operators & Built-in Functions

- **Relational**: =, !=, <, <=, >, >=
- **Arithmetic**: +, -, *, /, %
- **Logical**: AND, OR, NOT
- **String**: LIKE, RLIKE, CONCAT, SUBSTR, LENGTH, UPPER, LOWER, TRIM
- **Aggregate**: SUM, COUNT, AVG, MIN, MAX, STDDEV
- **Math**: FLOOR, CEIL, ROUND, ABS, SQRT

5.3 Managed (Internal) Tables

- Created by default (no `EXTERNAL` keyword)
- Hive **owns the data**; stored under warehouse path (e.g., `/user/hive/warehouse/dbname.db/tablename/`)
- `DROP TABLE` deletes **both metadata and data** from HDFS

5.4 External Tables

- Created with `CREATE EXTERNAL TABLE ... LOCATION 'path'`
- Hive manages **only metadata**; data resides in specified location
- `DROP TABLE` by default removes **only metadata**, not actual files
- Good for sharing data or data managed by external systems

5.5 Partitions

- Directory-level split on partition columns
- Created with `PARTITIONED BY (dt STRING, region STRING)`
- Data physically stored in subdirectories: `.../table/dt=2025-01-01/region=IN/`
- **Partition pruning**: queries filtering on partition columns can skip other partitions → major performance boost

5.6 Buckets

- File-level split inside partitions
- Created with `CLUSTERED BY (user_id) INTO 8 BUCKETS`
- Data hashed on bucket column, written to multiple bucket files
- Helps with sampling, map-side joins, query optimization

5.7 Storage Formats

- **Row-oriented**: TEXTFILE, AVRO (less efficient for analytical scans)
- **Columnar**: ORC, PARQUET (better compression, faster analytical queries)
- Columnar formats preferred for modern Hive analytics

5.8 Importing Data (DML Operations)

- `LOAD DATA INPATH '/path/file.csv' INTO TABLE t;` – Move/copy files from HDFS into table location
- `LOAD DATA LOCAL INPATH 'localfile' INTO TABLE t;` – From local file system
- External sources via Sqoop, Spark, or ETL tools

5.9 Altering Tables (DDL)

- Add partition: `ALTER TABLE t ADD PARTITION (dt='2025-01-01');`
- Add/replace columns: `ALTER TABLE t ADD COLUMNS (new_col STRING);`
- Change properties: `ALTER TABLE t SET TBLPROPERTIES (...);`
- Change file format: `ALTER TABLE t SET FILEFORMAT ...;`

5.10 Dropping Tables

```
DROP TABLE t;
```

- **Managed table**: drops metadata + data
- **External table**: drops metadata only; data files remain

6 SESSION 18: QUERYING WITH HIVE QL

6.1 Sorting Clauses

- `ORDER BY` – **Global sort** using a single reducer; entire result is totally ordered
- `SORT BY` – Sorts data **per reducer only**; overall output may not be globally sorted
- `DISTRIBUTE BY` – Controls which rows go to which reducer (by key)
- `CLUSTER BY` – Shorthand for `DISTRIBUTE BY` + `SORT BY` on same columns

6.2 Aggregation with GROUP BY

- Standard aggregate functions: SUM, COUNT, AVG, MIN, MAX, STDDEV
- `GROUP BY` groups rows by specified columns
- `HAVING` filters **groups after aggregation** (vs `WHERE` filters rows before grouping)

6.3 Joins in HiveQL

- Supported types: INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS, SEMI
- Basic syntax: `SELECT * FROM a JOIN b ON a.id = b.id;`
- **Reduce-side (common) join / shuffle join**:
  - All data for a join key shuffled to the same reducer, joined there
  - Works for arbitrary-sized tables; involves heavy network shuffle
- **Map-side join (MapJoin)**:
  - One table must be small enough to fit in mapper memory
  - Small table loaded on each mapper; mappers stream big table and join locally
  - No reduce phase; less shuffle → better performance
  - Enabled with hint: `SELECT /*+ MAPJOIN(small_table) */ ...`

6.4 Subqueries

- **Subqueries in FROM clause** (derived tables):
  - Pre-aggregate data before joining
  - Calculate top-N per group, then filter on that result
- **Subqueries in WHERE clause**:
  - Scalar subqueries for filtering based on another query result
- Combine filtering + aggregation in single logical step

6.5 Views

- **Virtual table** defined by a saved query; no separate data storage
- `CREATE VIEW top_regions AS SELECT region, SUM(sales) s FROM sales GROUP BY region;`
- Query like a table; underlying base tables are scanned
- Benefits: centralize logic; changes apply to all consumers; simpler interface for complex queries

6.6 MapReduce Scripts (TRANSFORM Clause)

- Integrate external scripts in MapReduce-like fashion
- Syntax: `SELECT TRANSFORM(col1, col2) USING 'myscript.py' AS (out1, out2) FROM t;`
- Script reads tab-delimited input from STDIN, writes output to STDOUT
- Hive handles parallelization and MapReduce plumbing
- Older syntax: `MAP ...` `REDUCE ...` clauses

7 SESSION 19: MORE ON HIVE QL (DATA MANIPULATION, UDFS, SCRIPTS)

7.1 Data Manipulation (DML)

- `INSERT INTO TABLE t SELECT ...` – **Append** data to existing table/partition
- `INSERT OVERWRITE TABLE t SELECT ...` – **Replace** existing data in table/partition
- `LOAD DATA INPATH 'hdfs_path' INTO TABLE t` – Move/copy files into table (append at file level)
- HDFS is write-once; updates/deletes done by rewriting partitions with `INSERT OVERWRITE`

7.2 User-Defined Functions (UDFs)

Types:

- **UDF**: Scalar function (row-by-row transformation)
- **UDAF**: Aggregate function (e.g., custom SUM-like behavior)
- **UDTF**: Table-generating function (expands one row into multiple rows)

Registration:

1. `ADD JAR /path/myudf.jar;`
2. `CREATE TEMPORARY FUNCTION myfunc AS 'com.example.MyUdf';`
   - Temporary functions live for the session only
   - Permanent functions stored in metastore (version-dependent)

7.3 Custom MapReduce in Hive

- Via `TRANSFORM`, `MAP`, `REDUCE` clauses
- Script acts as mapper/reducer: reads tab-delimited STDIN, writes tab-delimited STDOUT
- Hive distributes and manages MapReduce execution
- Allows any language (Python, Bash, etc.) for complex transformations

7.4 Appending Data into Existing Tables

- `INSERT INTO TABLE sales SELECT ...;` – Append into table
- `INSERT INTO TABLE sales_p PARTITION (dt='2026-01-05') SELECT ...;` – Append into partition
- `LOAD DATA INPATH '/data/file' INTO TABLE sales;` – Append files

7.5 Writing HQL Scripts (.hql Files)

- Put sequence of HiveQL commands in `.hql` file (each ending with `;`)
- Execute from shell:
  - `hive -f myscript.hql`
  - `beeline -f myscript.hql`
- Used to automate ETL jobs, daily reports, multi-step operations
- Provides reusable, version-controlled pipeline definitions
- Commonly scheduled in production via Oozie, Airflow, or other schedulers

7.6 Common HiveQL SELECT Clauses (Summary)

```
SELECT columns
FROM table
WHERE row_filter
GROUP BY group_columns
HAVING group_filter
ORDER BY / SORT BY / DISTRIBUTE BY / CLUSTER BY columns
LIMIT count;
```

8 KEY MCQ TRAPS & MNEMONICS

8.1 ETL vs ELT

- **ETL**: Extract → Transform → Load (traditional DWH)
- **ELT**: Extract → Load → Transform (modern data lakes; load all raw data first, transform on-demand)

8.2 Hive vs RDBMS in One Sentence

- **Hive**: "SQL queries on huge HDFS files executed as distributed batch jobs"
- **RDBMS**: "Row-level transactions on indexed local storage at interactive speeds"

8.3 Managed vs External Table on DROP

- **Managed**: `DROP TABLE` removes metadata **and** data → data gone forever
- **External**: `DROP TABLE` removes metadata only → data files remain accessible

8.4 Partitions vs Buckets

- **Partitions**: Directory-level split → prune entire directories in queries → huge performance gains
- **Buckets**: File-level split within partitions → helps joins/sampling, not query pruning

8.5 Reduce-side vs Map-side Join

- **Reduce-side**: Any table size; heavy shuffle (default)
- **Map-side**: One table must fit mapper memory; minimal shuffle (faster)

8.6 Sorting Clauses Ranking (Performance Impact)

- Fastest: `SORT BY` (per-reducer sort only)
- Medium: `DISTRIBUTE BY` (partition data, no sort)
- Slowest: `ORDER BY` (global sort, single reducer bottleneck)

8.7 Hive Operations Cheat

- **DDL (schema)**: CREATE TABLE, ALTER TABLE, DROP TABLE
- **DML (data)**: INSERT INTO, INSERT OVERWRITE, LOAD DATA
- **Query**: SELECT with WHERE/GROUP BY/HAVING/ORDER BY
- **UDF**: Custom Java logic registered via ADD JAR + CREATE FUNCTION

9 QUICK REFERENCE: ESSENTIAL COMMANDS

9.1 HBase Shell (Most Common)

```
hbase(main):001:0> create 't1', 'cf1'
hbase(main):002:0> put 't1', 'row1', 'cf1:col1', 'value1'
hbase(main):003:0> get 't1', 'row1'
hbase(main):004:0> scan 't1'
hbase(main):005:0> count 't1'
hbase(main):006:0> disable 't1' -> drop 't1'
```

9.2 Hive QL (Most Common)

```
CREATE TABLE sales (id INT, region STRING, amount DOUBLE);
LOAD DATA INPATH '/data/sales.csv' INTO TABLE sales;
SELECT region, SUM(amount) FROM sales WHERE amount > 0 GROUP BY region;
INSERT OVERWRITE TABLE sales_summary
  SELECT region, SUM(amount) FROM sales GROUP BY region;
CREATE VIEW high_sales AS
  SELECT region, amount FROM sales WHERE amount > 5000;
```

9.3 Hive MapReduce Script Integration

```
SELECT TRANSFORM(col1, col2)
USING 'myscript.py'
AS (output1, output2)
FROM input_table;
```

9.4 Execute HQL Script

```
$ hive -f queries.hql
$ beeline -f queries.hql
```

10 EXAM STRATEGY REMINDERS

1. **Always distinguish**:
   - ETL (traditional) vs ELT (data lake pattern)
   - Managed vs External tables (especially DROP behavior)
   - Reduce-side vs Map-side joins (default vs optimized)
2. **Data model clarity**:
   - Row keys are sorted lexicographically (HBase)
   - Partitions are directories (Hive)
   - Buckets are files within partitions (Hive)
3. **Scaling rules**:
   - HDFS scales via replication + rack awareness
   - HBase scales via region splitting
   - Hive scales via partitions (prune), buckets, and distributed execution engine
4. **Performance patterns**:
   - Map-side join > Reduce-side join (when small table fits memory)
   - Column-oriented formats (ORC, PARQUET) > row-oriented (TEXT) for analytics
   - Partition pruning filters before shuffle (huge savings)
5. **MCQ red flags**:
   - "Always" or "never" statements are often wrong
   - Details matter: `DROP TABLE` on managed vs external, `INSERT INTO` vs `INSERT OVERWRITE`
   - Multiple-correct-answer questions are common

11 FINAL SUMMARY TABLE

Component	Core Purpose	Key Concepts
ETL	Data pipeline design	Extract, Transform, Load; ELT pattern; Sqoop, Flume, Kafka
HBase	Fast random read/write NoSQL	Row keys, column families, regions, RegionServers, WAL, ZooKeeper
Hive	Batch SQL analytics on HDFS	Metastore, partitions, buckets, managed/external, MapReduce execution
Joins	Combine multiple data sources	Reduce-side (default), map-side (small table in memory)
UDFs	Custom logic	Java JARs, ADD JAR, CREATE TEMPORARY FUNCTION
Scripts	Automation	HQL files, `hive -f`, orchestration scheduling

Good luck on your CCEE exam!
Study the MCQ patterns, understand the core concepts deeply,
and remember the key differences between similar-sounding terms.